PRIVOKE: A Client-Side Semantic Privacy Firewall for User Prompts in Web-Hosted LLMs

Nicholas Bisset*
Monash University Clayton, Australia
nbis0006@student.monash.edu

Manpreet Sahadevan, Nixon Ngai, and Sanoop Mallissery
Monash University Malaysia, Malaysia
{msah0018, nnga0004}@student.monash.edu, sanoop.mallissery@monash.edu

***Abstract*—Useful prompts often contain context that a user may not intend to disclose to an external language-model provider. PRIVOKE places local inspection and explicit allow, warn or block actions before transmission on supported client paths. Its implemented lifecycle connects request enforcement to a separate training service, which checks version and retention constraints before distributing accepted detector parameters without an application release. Locally randomized telemetry reports independently and supplies no training data. We evaluate detection and enforcement, retention during updates, and systems and privacy overhead. Updated packages improved agreement with authored provisional contextual targets, but action errors remained and specificity on a separate annotation-presence task was only 13.36 to 21.76%. Installed-browser probes verified the declared request behavior on tested paths. Every matched full-model candidate improved aggregate agreement yet failed the retention guard. Cost measurements described the instrumented configuration without isolating incremental overhead; telemetry audits found a small excess over nominal privacy loss and substantial synthetic estimation error. These findings support an inspectable protection and revision mechanism while leaving independent contextual accuracy, useful adaptation without lost protection and practical user benefit unresolved.**

***Index Terms*—Prompt privacy, local inference, request enforcement, model regression, randomized telemetry, local differential privacy.**

I. INTRODUCTION

Web-hosted large language models support writing, coding and analysis by processing the instructions and contextual material that users provide [1]–[3]. A useful prompt may therefore contain much more than a question: a user seeking help with a report might include a colleague's details, health information or confidential organizational material. The same context that helps the model complete the task can consequently disclose information to an external provider. Reported workplace incidents illustrate how this can occur during otherwise legitimate work [4], [5]. Controls over subsequent storage or use do not prevent the initial transfer. Inspecting the request on the user's device provides an opportunity to intervene while the information is still under local control.

Deciding when to intervene requires more than recognizing names or identifiers. Two prompts can discuss the same subject while differing in whose information they contain, who is intended to receive it and whether that disclosure is appropriate. Nissenbaum's account of contextual integrity similarly relates privacy to the norms governing information flows within a particular context [6]. A detector can represent some of these distinctions, but its labels do not independently establish authorization. Its decision must also have an observable effect on the request: notifying a user that content may be sensitive is different from preventing that content from leaving the device. These considerations motivate evaluating contextual predictions, policy decisions and actual transmission separately.

Motivated by this problem, we present PRIVOKE, an implemented client-side privacy firewall for supported outgoing prompts. The local detector combines rules, named entity recognition and semantic classification, describing information through sensitivity, intended audience and categories before selecting ALLOW, WARN or BLOCK. The extension releases the original request for ALLOW and WARN and cancels a supported request for BLOCK. WARN attempts to display a notice without waiting for acknowledgement, so it does not establish consent or prevented disclosure. This explicit request behavior allows the evaluation to distinguish a correct classification from an intervention that actually stops transmission.

Maintaining this protection introduces a second problem: a revised model can improve average accuracy while becoming wrong on cases that its predecessor handled correctly [7]. PriVoke therefore separates training a candidate from accepting it for use. A server-side runtime learns from supplied labelled examples, and the update service checks the candidate against its exact parent model and declared retention criteria before publication. Clients verify accepted artifacts and refresh their local parameters without an application release. A third, independent path randomizes coarse event reports on the workstation. Neither intercepted prompts nor telemetry supplies training labels, allowing detector maintenance and reporting to be examined without treating collected user traffic as supervision.

Local protection and learned sanitization already have substantial precedent in Casper, Hide and Seek and ProSan [8]–[11]. Our contribution is the design and evaluation of a lifecycle that connects explicit request enforcement, guarded detector revision and separate randomized reporting. We examine whether the implemented mechanisms deliver their intended behavior, how improvements coexist with casewise losses, and what processing and reporting costs remain. This

*Corresponding author: Nicholas Bisset.
GitHub Repository Link

scope supports an engineering contribution even when an update is rejected or an evaluation exposes a limitation; it does not presume that local execution, successful training or a passing guard establishes effective privacy protection.

We organize the evaluation around three questions:

1) **RQ1: Does PriVoke detect and enforce privacy-sensitive prompt policies effectively?**
2) **RQ2: Can PriVoke update its detector without sacrificing previously protected behavior?**
3) **RQ3: What systems and privacy overhead does PriVoke introduce?**

RQ1 connects contextual predictions to the consequences of allowing or stopping a request. RQ2 asks whether improvements survive a retention requirement, rather than assuming that a newer model deserves publication. RQ3 examines the resources consumed and information released by protection itself. The following sections develop the prior-work basis and system design before answering these questions using the available evidence, including negative results and the limits of provisional contextual labels.

II. Related Work and Contribution

Local intervention and contextual decisions. Casper demonstrates how prompt protection can be placed on the user's device through a browser extension combining rules, entity recognition and local topic analysis [8]. Its reviewed prototype replaces identifiers with placeholders, supports user-triggered restoration in responses and requires acknowledgement of topic warnings. Analysis is initiated through a keyboard shortcut. Casper also permits custom rules and topics and compatible model replacement, establishing precedent for both layered local inspection and configurable protection. PriVoke builds on that motivation while examining a different request contract: its supported hooks automatically apply a decision, but a WARN releases unchanged content without waiting for the user.

Hide and Seek (HaS) addresses the usefulness of protected interactions by hiding entities before cloud inference and recovering them in the response [10]. Its generative variant trains hiding and recovery models, while the label-based variant uses an entity recognizer for hiding. ProSan instead uses task-importance and privacy proxies to select word replacements, then trains a sanitizer for local execution [11]. These approaches show that learned local protection can include both transformation and recovery. PromptShield considers a related enterprise setting through role/policy classification [12]. Together, these works motivate separating the detection of sensitive information from the decision about its disclosure and the user's ability to complete a task. PriVoke's sensitivity, audience and category labels provide a representation for its policy; they do not constitute an independently validated model of contextual authorization.

Retaining behavior during model revision. The distinction between better average performance and preserved individual predictions is established in model-update research. Yan et al. study *negative flips*, where an updated image classifier becomes incorrect on a previously correct example, and introduce positive-congruent training to reduce such regressions [7]. Their result motivates measuring both gains and losses, although image-classification evidence does not establish privacy-policy correctness. PriVoke examines the related engineering decision of whether to publish a candidate: label and action outcomes are compared with the declared base, and a configured retention guard can reject an otherwise improved model. Binding that decision to a versioned artifact and verified client refresh makes acceptance distinct from successful optimization. The contribution is this implemented and evaluated connection; neither casewise regression analysis nor model training is claimed as new, and the guard covers only its declared cases and criteria.

Privacy of operational reporting. Local randomization provides a foundation for collecting aggregate information without sending each underlying value directly to a collector. RAPPOR applies randomized response to client-side statistics and explicitly addresses the additional privacy cost of repeated reporting [13]. PriVoke uses categorical event reports with a local budget ledger, rather than adopting RAPPOR's memoized reporting mechanism. Its relevant question is therefore whether the implemented sampler and accounting satisfy their stated event-content bounds, and whether the resulting estimates are useful. We examine finite arithmetic, estimation error and contribution selection separately. The reporting mechanism does not protect participation or network metadata, and aggregate event fields supply neither corrected contextual labels nor evidence that the detector should be retrained.

Contribution and comparison boundaries. These strands lead to three distinct evaluation needs: establish the relationship between a detection and its request outcome; assess a revision through retained behavior as well as aggregate gain; and account for the resources and disclosure introduced by the protection mechanism. PriVoke brings them together in one implemented lifecycle. It does not replace outgoing text with its returned masked spans or recover response entities, so sanitization utility cannot be inferred from its request decisions. Likewise, Casper's entity/topic measures, HaS's whole-text recovery resistance and ProSan's item-replacement and task-utility measures have different targets and denominators. Supporting Detail, Section D preserves those version-qualified comparisons. Our executed Presidio configurations provide entity-recognizer references [14], not matched reproductions of those systems. The resulting contribution is a measured account of enforcement, retention and reporting tradeoffs, with broader contextual accuracy and practical benefit left to evidence that the present studies do not supply.

III. System Design and Privacy Boundary

A. Scope and Trust

PriVoke is designed to intervene when personal, third-party or organizational information is about to leave the workstation through a supported prompt request. Its protection depends on the enabled extension, local runtime and supervisor, workstation and authenticated model publisher

operating as configured. These components form the trusted boundary. A compromised client, malicious publisher or page that deliberately replaces the interception hooks can undermine that boundary, while information already held by a provider cannot be protected by stopping a later outgoing request.

The implemented interception path recognizes prompt POST formats submitted through page `fetch` and asynchronous `XMLHttpRequest`. Workers, attachments, other transports and changed endpoints require their own validation; support for the two hooks does not establish coverage of those paths. Users can also disable protection, which bypasses inspection by design. An optional hosted-inference configuration sends text beyond the workstation and therefore has a different disclosure boundary. It is excluded from the local-inference measurements in this paper.

Within this boundary, Figure 1 separates three kinds of information flow. Prompt text travels from the browser to the workstation detector and reaches the hosted LLM only if the request action permits release. Supplied labelled examples follow a separate training path, from which accepted model parameters return to clients. Coarse event fields follow a third path through local randomization to a telemetry collector. The separation makes each service's input explicit: maintaining the detector does not require collecting intercepted prompts, and the collector supplies no training labels. Whether the detector makes an appropriate decision remains an empirical question.

B. Local Inspection and Request Enforcement

Inspection begins when a page hook extracts the prompt from a supported outgoing request. The extension's background process sends the text through a loopback gRPC-Web bridge to the workstation's Python runtime. Native messaging starts the supervisor that manages the bridge and runtime; it does not send the prompt to the separate server-side training runtime. After analysis, the hook uses the returned action to release or cancel the original request (Supporting Figure S1). The training instance is not a remote fallback for local user inference.

The runtime first normalizes the text while retaining offsets that connect its findings to the original input. It then evaluates regular-expression rules before the enabled named entity recognition (NER) and semantic layers. A rule-based BLOCK ends the cascade immediately. Otherwise, the findings are combined using the strongest action. This ordering has a direct consequence for interpretation: a semantic classifier cannot reverse an earlier rule-based restriction, and an early BLOCK means that the later requested detectors did not run.

Failure handling follows the same need to identify what was actually assessed. A failed requested layer changes an otherwise ALLOW result to BLOCK, while valid non-ALLOW findings remain visible. If analysis itself is unavailable, the extension blocks supported requests. Losing the parameter stream can instead leave rules and NER available as a reduced detector configuration. That fallback may release a request despite losing semantic coverage, so a returned action must be interpreted together with the recorded layer execution.

The semantic classifier represents a prompt through sensitivity, audience and information categories. Sensitivity S0–S3 distinguishes no identified risk, low-risk personal context, potentially harmful personal information and highly sensitive information. Audience, called *visibility* in the implementation, distinguishes public, semi-public, authentication-restricted, group-private, personal-private and unspecified contexts (P0–P4 and PU). A prompt can carry several of ten categories, including health, identity and location. The policy normally assigns BLOCK to S3 and WARN to S2, with identity/location combinations able to increase restriction. A returned confidence score below 0.5 can moderate the action; Supporting Detail, Section A specifies the complete mapping. This score summarizes the sensitivity/category heads and is not a calibrated probability of full correctness or authorized disclosure. The labels and mapping are therefore policy inputs whose validity must be evaluated, rather than evidence of authorization in themselves.

The models performing this task are classifiers, not conversational response generators. The evaluation compares historical small classifiers with later packages retaining pretrained BERT encoders and poolers [15]–[18]. Both generations offer efficient, balanced and quality packages. These names identify configurations rather than a guaranteed performance ordering. Tokenizers, capacities, thresholds and training histories differ as well, so the comparison measures the resulting package change rather than the isolated effect of pretraining or architecture.

C. Guarded Detector Updates

The update path asks two separate questions: whether a candidate belongs to the intended model, and whether its behavior meets the configured acceptance criteria. A controller prepares bounded labelled batches and held-out guard cases, excluding protected rows, texts and groups from training. The server runtime loads the specified base, trains a candidate and returns parameter changes and metrics tied to that base. Before publication, the updater checks base identity, permitted tensors and shapes, finite values, magnitude limits and the strategy's configured guards (Supporting Figure S2). Artifact checks establish that the candidate is well formed; retention checks assess its behavior on declared cases. Completing a fit or transferring its parameters cannot substitute for either decision.

An accepted candidate is published through an atomic compare-and-swap operation, which requires the current base still to match its declared parent. This prevents a decision made for one model from silently replacing a different intervening revision. Durable replay receipts associate an identical retry with its earlier artifact outcome instead of applying it twice. Head and underlying-model stages are separate publications, however, so rejecting a later stage does not undo an accepted head update. Clients receive versioned parameters from the streaming service and verify artifact identities before refreshing cached weights, allowing model delivery without an application release.

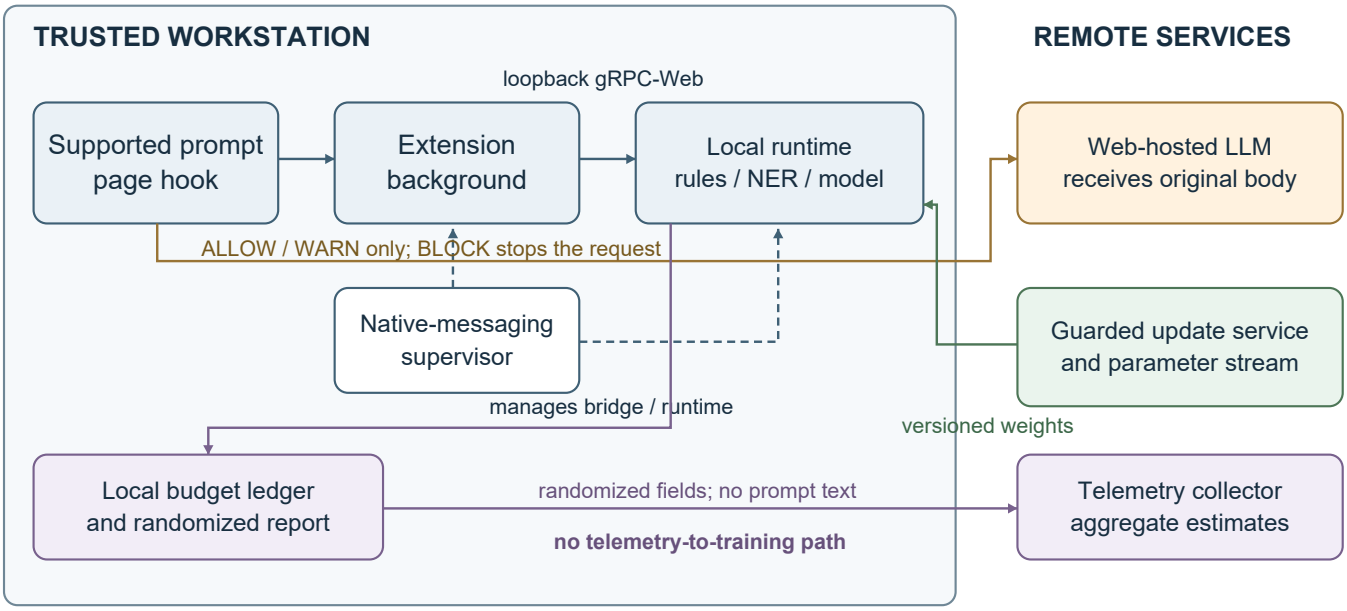


Fig. 1. PriVoke's three data paths. Supported prompt text is inspected inside the trusted workstation. Only an ALLOW or WARN decision releases the original body to the web-hosted LLM. Native messaging manages the local bridge/runtime; analysis uses loopback gRPC-Web. A separate update service supplies versioned weights, and a separate collector receives randomized event fields. Training consumes supplied labelled examples (Supporting Figure S2), not intercepted user prompts or telemetry. Dashed lines denote lifecycle control.

This design supports centralized training from supplied examples while keeping user inference local. It neither collects user gradients nor implements federated learning, and an implemented service path does not establish cloud deployment. RQ2 accordingly separates the ability to compute, publish and receive a revision from evidence that the particular revision should be retained.

D. Separate Randomized Telemetry

Telemetry describes coarse aspects of an event through five fields: action, an action/sensitivity risk bucket, one category selected by fixed precedence, a bounded release label and a four-hour time-of-day bin without a date. The workstation randomizes each field before release. Reports omit raw prompts, identifiers, exact confidence scores and per-layer findings, reducing the information presented to the collector without making the retained fields intrinsically harmless. The collector estimates event marginals and supplies no corrected labels or training inputs.

The intended local differential privacy guarantee limits how much the probability of an output can change between possible event contents [19]. Here the comparison holds the reporting opportunity and public configuration fixed; it does not hide whether an event was reported. An event budget ε is divided equally, giving each field $\varepsilon_i = \varepsilon/5$. A field with K_i possible values reports its true value with probability p_i and each alternative with probability q_i:

$$p_i = \frac{e^{\varepsilon_i}}{e^{\varepsilon_i} + K_i - 1}, \quad q_i = \frac{1}{e^{\varepsilon_i} + K_i - 1} \tag{1}$$

With independent ideal samples, multiplying the field likelihood ratios gives an event bound of e^{ε}, following composition [19]. Finite arithmetic can change those probabilities, so the implemented bound requires its own audit. A persistent ledger reserves budget before report construction, with defaults of one per event and eight per UTC day; construction or delivery failure does not refund a reservation. Retained state and a trusted clock are necessary for this accounting. Report presence, contribution selection, delivery and network metadata remain outside the content guarantee. Inverting the nominal response probabilities and clipping the resulting marginal estimates introduces a separate utility question, examined alongside the privacy bound in RQ3.

IV. Evaluation Method

A. Questions and Evidence Units

The evaluation follows the consequences of a decision from prediction to request disposition, while treating each stage as a separate outcome. Table I connects the available evidence to the three research questions. Several analyses reuse the same cases, so their agreement does not provide independent confirmation. Detector scope is also explicit: semantic-only calls request that layer alone and verify its actual execution, whereas product calls request rules, NER and semantics and record legitimate skips after an early rule-based BLOCK. Requested layers therefore do not automatically count as executed layers.

TABLE I
EVIDENCE UNITS AND THEIR LIMITS. PRESENCE/CATEGORY LABELS ARE AVAILABLE PROXIES; INDEPENDENTLY ADJUDICATED CONTEXTUAL PRIVACY TRUTH IS MISSING.

RQ	Evidence	What it establishes
1	1,560 authored contextual cases; 498 final presence rows; 2,000 external category rows	Agreement with distinct targets; not human privacy accuracy.
1	33 installed-browser receiver probes	Bounded request disposition; not provider coverage or usability.
2	Paired package outcomes; matched head-only/full-model continuation	Casewise gains/harms and guard decisions; not guaranteed adaptation.
3	360 browser decisions in 12 profiles; finite telemetry audit and synthetic simulation	Scoped systems cost and conditional privacy/estimation behavior.

Contextual targets and measures. The authored workload contains 1,560 cases: 810 sensitive and 750 clean, with 260 in each audience class. Sensitivity, audience, category and allowed-action targets were prepared provisionally with assistant support and have not been independently human-adjudicated. Agreement with these targets measures consistency with the authored policy, not established disclosure authorization. For binary scoring, a nonzero sensitivity label or nonempty category set defines a positive.

We distinguish two joint measures to avoid conflating labels with interventions. *Label-joint agreement* requires exact sensitivity, audience and category-set matches. *Label-and-action joint agreement* additionally requires an allowed action; both the product comparison and the semantic-only matched-continuation assessment use this latter measure. Action agreement can also be measured alone. An action below the allowed set is under-restrictive, while one above it is over-restrictive. Whether the original content actually reached a receiver is a separate systems observation, not another classification score.

The cases span 35 scenario families and 24 audience-expression groups. They were sealed for an earlier semantic comparison and subsequently opened and reused for product and recognizer studies. Those later comparisons are therefore development evidence on PriVoke-authored material, even where their procedures were fixed prospectively. Repeated passes test whether the same inputs produce the same outputs and do not enlarge the independent sample. For contextual product binary measures, we used 2,000 paired whole-cluster draws, seed 42 and linear 95% percentile ranges, separately by family and audience expression. These analyses describe sensitivity to the declared dependencies; they do not establish crossed-cluster independence or population coverage.

Checks against different targets. The once-only final endpoint comprised 498 English rows from PIIMB revision `4a13e9f`: 236 annotation-positive and 262 annotation-negative targets [20]. Models, source inputs and scoring were frozen before release, and the results were followed by no tuning, reselection or promotion. This endpoint asks whether a prompt contains a source annotation, which differs from contextual authorization and from PIIMB's official character-masking/F2 task. Its 464 observed source-group keys describe the grouping convention without certifying independent templates.

A separate AdvPIIBench reference used 2,000 rows: 1,000 injected positives, 750 ordinary negatives and 250 hard negatives [21]. These official categories identify the source's injected targets rather than exhaustively annotating privacy throughout a prompt. This reference also used a different prediction projection, counting any semantic finding as positive. Its scores therefore cannot be pooled with the nonzero-sensitivity-or-category measure. Supporting Detail, Section B retains the selection rules, study-specific grouping and prediction definitions needed to interpret both checks.

B. Comparison and Retention Protocols

Historical and updated packages were evaluated through the product handler on the same contextual cases, with rules and policy held fixed. Pairing reveals which cases improve or regress, but the packages differ in more than their fitted weights. Historical efficient/balanced/quality input windows were 64/96/128 tokens with category threshold 0.38; updated packages used 512 WordPieces and threshold 0.5. Together with capacity, tokenizer and training-history differences, these settings prevent attributing the package contrast to a single model property. A narrower balanced-package ablation fixed texts, weights and policy while removing individual detectors, isolating their contribution within that configuration.

Configured small and large English Presidio models supplied same-text entity-recognizer references at threshold 0.5 [14], [22]. They did not produce the contextual labels and actions required by PriVoke's complete policy, so those unsupported outputs were left unmeasured. The restricted external-category reference used a separate Presidio configuration, detailed in Supporting Detail, Section B; it is not the same baseline under a different dataset name.

RQ2 used a fixed-base comparison of head-only and full-model continuation across three seeds. Each treatment started from byte-identical intact pretrained balanced encoder/pooler weights and zero task heads, received the same ordered 4,096 training exposures over 64 steps, and was assessed on 1,080 reused provisional cases after complete training/guard group and text exclusions. Full-model continuation alone enabled encoder optimization, which also changed joint-gradient clipping. Every terminal candidate was assessed, including guard failures; assessment outcomes did not select a checkpoint. The comparison thus tests the two optimization treatments under matched exposure, without turning the reused cases or repeated seeds into independent populations.

The 16-case retention guard required improved joint correctness and zero action harms against a single derived target action for each case. The broader 1,080-case assessment instead accepted its declared sets of allowed actions. Passing the

guard and improving assessment agreement therefore answer different questions, using different cases and action criteria. The small guard and weak zero-head control further limit what either outcome establishes. No candidate in this matched study was published: effective-artifact mappings describe what the guard would retain. Supporting Detail, Sections E and B preserve the training recipes, selection history and unsuccessful branches.

C. Enforcement, Cost and Telemetry Protocols

The installed-browser study followed requests through the built extension in Linux Chromium, using the standard native-host installer, native messaging, supervisor, bridge and historical balanced detector. An HTTPS synthetic receiver recorded arriving original bodies and checked for late sends. Thirty-three correctness probes exercised forwarding, blocking, faults, cancellation and recovery, connecting the expected action to an observable request outcome. Because the receiver was synthetic rather than an authenticated LLM provider, the study establishes behavior on the declared paths. Full-pipeline and sole-semantic execution checks took place outside the resource-measurement windows.

The cost experiment measured 360 rules/NER decisions across twelve fresh profiles, with two profiles per transport/action cell. Latency ran from the page's analysis request to its analysis result, excluding the receiver response. CPU and sampled charged memory covered the whole container, including resident infrastructure, cache and the observer. These boundaries matter: the experiment included neither semantic processing nor an unprotected baseline or per-process attribution. Its results describe the instrumented system rather than incremental PriVoke overhead. Two profile clusters per cell also limit interpretation of pooled latency tails; Supporting Detail, Section F specifies the observation and sampling windows.

The telemetry study examined the mechanism, estimator and accounting separately. Its audit enumerated finite sampling kernels for one fingerprinted host build, assuming independent uniform OS bits. Fixed-contributor simulations measured estimation error across 180 cells with 10,000 repetitions each, while real ledger checks exercised persistence, concurrency and failure accounting. The simulation's target was the histogram of contributing events, not an independently sampled deployment population. These procedures establish conditional mechanism and synthetic-estimation evidence; they do not measure deployed monitoring usefulness.

V. RQ1: Detection and Enforcement

A. Contextual Policy Agreement

The updated product packages more often selected the authored labels and an allowed action together. Label-and-action joint agreement increased by 27.37–41.86 percentage points, reaching 693–879 of 1,560 cases (Table II). All six runs completed without failed calls or layer errors. Supporting Figure S3 shows the accompanying component and enforcement outcomes, which help explain what the aggregate improvement means. Efficient, balanced and quality gained 484/573/671 previously incorrect joint outcomes while losing 57/16/18 previously correct ones. Allowed-action gains were 389/389/348, with losses of 3/6/0. Quality therefore preserved every previously allowed-action match while losing eighteen label-and-action joint matches: preserving an intervention does not necessarily preserve its classification. These are improvements on reused provisional targets, and no package was promoted.

TABLE II
Full product on the same provisional contextual cases. Entries are historical/updated counts. Labels+action denotes label-and-action joint agreement; Action denotes allowed-action agreement (both out of 1,560). Sensitive and Clean are correctly classified counts out of 810 and 750, respectively. Every arm had zero errors.

Package	Labels+action	Action	Sensitive	Clean
Efficient	266/693	1029/1415	544/808	494/629
Balanced	160/717	1026/1409	771/802	318/631
Quality	226/879	1073/1421	598/810	468/641

The execution records locate one important source of unnecessary restriction in the cascade. All updated packages over-restricted the same 139 cases: rules blocked 109 clean controls before later detectors could run and warned on thirty low-risk controls. Rules ran on every case and early-blocked 289 in total, leaving NER and semantics to execute on 1,271. The shared failures therefore could not be resolved simply by substituting a better semantic model behind those rules. NER contributed zero converted PriVoke findings after supported-label filtering. That observation concerns the findings admitted into the policy; it does not imply that the loaded recognizer was inactive or detected no raw entities.

Holding the balanced package and workload fixed allowed the detector-removal study to test this explanation. Removing rules raised allowed-action agreement from 1,409 to 1,530 and label-and-action joint agreement from 717 to 972. All 139 shared unnecessary restrictions were corrected, but eighteen previously correct actions were lost and under-restriction rose from twelve to thirty cases. The net gain of 121 actions thus came with additional missed protection under the provisional policy. Existing paired resampling placed the action-rate gain at 3.46–12.70 percentage points by family and 5.96–9.23 by audience expression. These separate descriptive ranges support the direction of the aggregate tradeoff without making the losses acceptable. Removing NER left full-product outputs unchanged, consistently with its zero converted findings on this fixture. All 10,920 calls retained valid execution records; neither removal result establishes that a detector is unnecessary on other workloads.

B. What Transfers Beyond the Authored Cases?

The final annotation-presence endpoint tested whether the same packages distinguished a different set of positive and negative targets. All 2,988 semantic-only calls completed without errors and verified sole execution of the requested

TABLE III
FINAL SEMANTIC ANNOTATION PRESENCE: 236 POSITIVE/262 NEGATIVE ROWS. HISTORICAL/UPDATED COUNTS; TRUE/FALSE POSITIVES (TP/FP), TRUE/FALSE NEGATIVES (TN/FN). ALL SIX ARMS HAVE ZERO ERRORS.

Package	TP	TN	FP	FN
Efficient	140/218	61/45	201/217	96/18
Balanced	141/221	65/35	197/227	95/15
Quality	167/190	51/57	211/205	69/46

layer. Updated recall reached 80.51–93.64%, while specificity remained only 13.36–21.76% (Table III). Efficient and balanced lost specificity relative to their historical versions; quality's small increase remained uncertain. Frozen source-group resampling nevertheless gave paired net-accuracy gain ranges of 7.41–17.37, 5.35–15.03 and 1.02–10.71 percentage points, respectively. The apparently conflicting observations have a straightforward explanation: enough additional positives were recognized to improve average accuracy despite poor discrimination of annotation-negative rows.

Those updated artifacts had achieved 98.40–99.87% specificity on the authored semantic-only contextual fixture. Their identities and the nonzero-sensitivity-or-category response projection remained fixed across the two endpoints; the texts and targets changed. The contrast is therefore evidence of a limit in transferring the earlier result, rather than an intervening change to the models. Distribution and target differences have not been isolated, so neither can be assigned sole responsibility. The poor final specificity still matters: strong agreement with authored clean controls did not establish broader discrimination. Neither endpoint, however, supplies independently adjudicated truth about contextual disclosure authorization.

The external source-category check exposed a similar trade-off using any semantic finding as the positive response. Balanced recall rose from historical 63.0% to updated 92.9%, while source-negative specificity fell from 48.6% to 0.5%. Restricted five-type Presidio showed the opposite pattern, with 99.1% specificity and 3.9% recall. All 6,000 model/recognizer analyses completed without errors, and no threshold or row was changed afterwards. These outcomes establish poor agreement with particular parts of the injected-category target, rather than a general ranking of privacy protection. In particular, the labels do not exhaustively describe privacy throughout each prompt, so 0.5% specificity is not an established 99.5% broad-privacy false-alarm rate.

Changing the recognizer on the authored contextual task likewise changed which cases were correct. Small-English Presidio achieved 45.31% recall and 71.87% specificity; the larger configuration reduced recall to 29.63% while increasing specificity to 88.80%. Both classified 906 of 1,560 cases correctly, because the larger model gained 192 correct cases and lost 192 others. Supporting Figure S4 places the small-model reference beside the updated product packages, whose recall was 99.01–100.00% and specificity 83.87–85.47%. The comparison makes their different operating behavior visible on identical texts. PriVoke-authored targets and the entity/contextual task mismatch prevent extending that observed advantage into a general privacy ranking (Supporting Detail, Section C).

C. Does a Decision Control Transmission?

Classification matters to enforcement only if the resulting decision changes what leaves the client. The completed installed-browser capture tested that connection across thirty-three declared correctness probes: the receiver observed eighteen unchanged-body sends and fifteen zero-send outcomes. Correlated runtime calls, receiver records and late-arrival checks supported those outcomes without a decision broker. Healthy full-pipeline and sole-semantic probes also verified actual execution, tying the observed request disposition to the tested detector path.

Fault probes showed why availability of a request path must be distinguished from preservation of its protection. Interrupting the parameter stream blocked semantic-only requests, whereas configured full-mode fallback ran rules and NER and forwarded the request. Restarting restored full execution; disabling protection or all layers forwarded requests by design. Fallback therefore maintained a reduced analysis path, but the capture does not establish equivalence to semantic protection. These completed observations remain separate from nine earlier failed captures, which were not pooled with them.

Answer to RQ1. PriVoke improved agreement with provisional contextual policy and enforced the declared request contract on the tested installed paths. The evidence supports those two bounded achievements, while action errors, original-content forwarding under WARN and weak specificity on separate presence/category tasks limit the effectiveness claim. Independent contextual accuracy, provider compatibility, prevention on every sensitive prompt and practical usability remain unestablished.

VI. RQ2: UPDATES AND PREVIOUSLY PROTECTED BEHAVIOR

A. Mechanism Versus Useful Adaptation

The update mechanism succeeded at changing served parameters through audited server computation, bounded publication and client refresh. Useful adaptation requires more: the changed detector must improve the intended behavior while retaining the protection required for acceptance. Earlier contextual, curriculum and accelerated training studies failed their declared qualification criteria. A pretrained package improved provisional disclosure agreement but still failed a serious-action veto. These unsuccessful branches, with their attempted budgets, remain in Supporting Detail, Section C; they establish that a functioning update path alone does not produce a qualifying detector.

Later supervised packages did improve authored contextual classification, although component and action regressions remained. Because their capacity, tokenizer, threshold and supervision changed together, the RQ1 comparison cannot identify which change produced the gains or assign them to automatic online training. The matched continuation study

TABLE IV
MATCHED SEMANTIC-ONLY CONTINUATION ON 1,080 REUSED PROVISIONAL CASES. ENTRIES ARE HEAD-ONLY/FULL-MODEL COUNTS. LABELS+ACTION DENOTES LABEL-AND-ACTION JOINT AGREEMENT. THE UNCHANGED ZERO-HEAD BASE HAS ZERO SUCH MATCHES AND 480 ALLOWED-ACTION MATCHES. GUARD HARMS COMPARE EACH CANDIDATE WITH THAT BASE ON SIXTEEN SEPARATE CASES.

Seed	Labels+action	Allowed action	Guard harms
42	192/257	908/993	0/1
43	204/247	906/989	1/1
44	198/268	912/1003	0/1

instead fixes the initial model and training exposure while varying whether the encoder is optimized. It allows the effect of those two declared optimization treatments to be examined alongside the cases each treatment loses.

B. Matched Continuation and Guard Decisions

All six 64-step fits completed with finite parameters. Head-only training preserved every encoder tensor; full-model training changed 39 and recorded nonzero later encoder gradients. The comparison therefore exercised the intended difference in optimization, rather than merely different treatment labels. The six candidates and unchanged base then completed 7,560 native in-process semantic assessments without errors. Each of these seven snapshots saw the same 1,080 situations, so the call count measures completed assessment work rather than independent cases or deployed network operation.

Full-model continuation achieved more label-and-action joint matches and allowed-action matches than head-only training in every seed (Table IV). For seeds 42/43/44, paired joint gains of 79/69/90 were offset by losses of fourteen, twenty-six and twenty cases; action gains of 123/124/110 were offset by thirty-eight, forty-one and nineteen losses. Full-model continuation also increased under-enforcement distance on 33/37/15 cases and over-enforcement distance on 17/13/17. The latter counts measure different directional changes and cannot be added into a mutually exclusive harm total. Together, these observations show why a higher aggregate score is insufficient evidence that previously correct protection has been retained. Three seeds on the same reused assessment do not establish the frequency of this tradeoff in an independent population.

The publication decision used a different comparison: each candidate against its unchanged base on the sixteen guard cases. Every full-model candidate introduced one action harm and failed that fixed rule, despite its better aggregate assessment. Head-only seeds 42/44 passed, while seed 43 failed. A failed candidate would therefore leave the base effective; it would not make the better assessment score available to clients. No candidate in this experiment was published or promoted, so these mappings describe hypothetical retention rather than deployed updates. The guard detected the losses represented by its cases and criteria, but a pass cannot rule out other harms or overcome the weak zero-head baseline.

Answer to RQ2. PriVoke can bind a revision to its base and reject candidates that violate the configured retention test. Useful improvement without sacrificed protection is not established: every matched full-model candidate improved aggregate agreement yet failed the guard, and broader package gains retained casewise regressions. The guard makes an acceptance boundary enforceable; the quality of supervision, coverage of retained cases and chosen criteria determine what protection that boundary can support.

VII. RQ3: SYSTEMS AND PRIVACY OVERHEAD

A. Measured Systems Cost

The installed-browser measurements describe the cost of reaching a local rules/NER decision. Across 360 decisions in twelve fresh-profile windows, median page-observed latency ranged from 27.9 to 35.0 ms by transport/action cell. All 240 ALLOW/WARN decisions ran both detectors, whereas the 120 BLOCK decisions ended after rules and legitimately skipped NER. The timing difference between these paths therefore includes different amounts of work. Each reported p99 was the observed cell maximum, rather than an independently estimated tail interval. Because the measurement ends at the analysis result, before the receiver response, it does not include downstream completion or semantic-layer processing.

The broader observation windows covered 126.40 seconds and accumulated 193.29 CPU-seconds across the container, including 0.69 observer CPU-seconds. CPU-seconds sum processor use across concurrent work and are distinct from elapsed time. Maximum sampled charged memory was 1,267.91 MiB, including resident services and cache as well as the driver and receiver. The maximum sampling gap of 622.84 ms leaves shorter memory spikes potentially unobserved. Final-read and cleanup checks passed, but neither isolates the extension and detector from the infrastructure being measured. These are costs of the instrumented configuration; without an unprotected baseline or per-role measurements, they cannot establish incremental PriVoke overhead. Startup intervals and cell tails are retained in Supporting Detail, Section H.

A separate product comparison observed lower host-call medians alongside larger server-process peak RSS for updated packages (Supporting Detail, Section G). That result identifies an observed tradeoff, but fixed execution order, warming, different backends and cascade shortcuts prevent attributing it to a controlled model speedup. The training records require the same care: ordered fit times do not establish comparative training speed, and allocated CUDA tensor peaks do not measure whole-process memory requirements. Combining these different observation boundaries would obscure rather than resolve the missing incremental-cost comparison.

B. Privacy Cost and Telemetry Utility

Auditing the finite implementation changed the privacy statement that could be made from the ideal formula alone. On the fingerprinted host, maximum event privacy loss exceeded nominal budgets 0.5, 1 and 2 by approximately $(1.294, 1.503, 1.226) \times 10^{-15}$. The excess is numerically tiny,

but an exact nominal guarantee does not hold for that audited kernel. Eight default reservations compose to approximately $8+1.202\times10^{-14}$ under the same independent uniform-entropy assumptions. Supporting Detail, Section I gives the finite calculation and host conditions. Its scope is the content of an admitted event under those conditions, rather than certification of other builds or deployment entropy.

The utility simulations ask a separate question: whether a histogram can be recovered accurately from the randomized reports. Inversion followed by clipping produced substantial bias and root-mean-square error, while separately estimated marginals often failed to form a coherent total. With eleven categories, event budget one and eighty contributors, a true first-category count of seven had clipped bias +22.84 and RMSE 42.32 counts. The estimated marginals failed to sum to eighty in 99.99% of trials. Error of this scale shows that the tested estimator cannot simply be treated as a reliable count of the synthetic contributors. Population monitoring accuracy and an acceptable deployment error threshold were not evaluated.

Ledger checks established a different property: the daily cap survived persistent state and concurrent requests, and construction or delivery failures still consumed reservations as required. This accounting bounds admitted contributions but also selects which events are represented. Reversing a constructed event sequence changed BLOCK participation among the first eight reports from 100% to 0%. Content randomization cannot correct that selection effect, and it does not protect whether an event participated. A bound on admitted report content therefore leaves both sampling representativeness and participation privacy unresolved.

Answer to RQ3. The tested configuration has measured local decision and infrastructure costs, while incremental overhead and semantic-pipeline cost remain unmeasured. Telemetry has an audited conditional event-content privacy loss slightly above its nominal budget, substantial synthetic estimation error and a separate contribution-selection effect. These findings quantify limits of the implemented reporting path without establishing useful deployment monitoring or protection of participation and network metadata.

VIII. Discussion and Limitations

A. What the System Evidence Supports

Privacy protection depends on how decisions are connected as well as how accurately a model classifies a prompt. An early rule BLOCK prevents semantic contribution, a policy-correct WARN releases original content, and an improved candidate remains unavailable if it fails retention. PriVoke makes these dependencies inspectable through execution records, version-bound candidates and separate reporting. Its contribution is this evaluated lifecycle, which exposes where detection, request handling and revision acceptance diverge without treating one successful component as proof of overall protection.

Local sanitization clarifies the practical question left open. Casper couples acknowledged warnings with restoration, HaS recovers hidden response entities, and ProSan evaluates utility after selective replacement [8], [10], [11]. These functions address task continuation after intervention. PriVoke neither rewrites prompts nor restores answers, so its request contract cannot establish the same benefit. An extension would need to evaluate authorized task completion directly.

B. Ground Truth and Generalization Remain Open

Contextual integrity concerns appropriate information flows [6]; entity recognition supplies only part of that judgment. Independent contextual ground truth remains the largest scientific gap because PriVoke's sensitivity, audience, category and action targets are assistant-provisional. Neither agreement with them nor agreement with annotation-presence or injected-category labels establishes authorization or human agreement. High authored clean specificity and poor final annotation-negative specificity both matter, despite their different targets. Entity identification, disclosure appropriateness, intervention choice and task utility each require suitable evidence.

Source-group keys and lexical exclusions track declared relationships without certifying template independence, covering paraphrases or removing unknown upstream pretraining exposure. The final endpoint is unevenly distributed across 308 Nemotron, 89 AI4privacy, eighty Gretel, twenty Privy and one MAPA row. These English synthetic or machine-generated workloads do not represent other languages or organizational policies. Cluster sensitivities remain descriptive and unadjusted; repeated runs and three seeds are not independent populations, and native-package differences constrain causal attribution. Learning from the opened final endpoint requires a new protected successor for confirmation.

C. Retention, Availability and Practical Utility

The gains and losses illustrate the update problem studied by positive-congruent training: higher average accuracy can coexist with newly incorrect cases [7]. PriVoke's acceptance guard addresses this problem only within its cases and criteria, a limitation amplified by its small size and weak zero-head control. Rejected candidates and earlier unsuccessful studies supply no served improvement. Stronger evaluation requires common pinned bases, independently reviewed retention cases and explicit casewise harm criteria, with failed candidates kept distinct from effective artifacts.

Blocking unavailable analysis can prevent disclosure while interrupting an authorized task; WARN instead proceeds without informed acknowledgement. Heuristic confidence is not calibrated joint correctness. The installed Linux evidence covers declared Chromium/native receiver paths with the historical detector, leaving authenticated providers, Windows, updated-model installation, workers, attachments and other transports unverified. Authorized task completion, unnecessary-intervention cost, warning comprehension and reformulation effort were unmeasured, preventing a claim of practical utility or human benefit.

Ephemeral events for notice rendering and local request disposition offer a measurement starting point. Fifty-three synthetic host tests validated their contract, but they postdate the browser capture and lack installed-browser or overhead

remeasurement. They are neither persisted nor sent as telemetry; page scripts can forge or suppress them. Rendering does not prove comprehension, nor native handoff server receipt. Utility evaluation still needs independently specified expected actions joined to user or task outcomes.

D. Measurement and Reproduction Boundaries

Container infrastructure and instrumentation, together with rules/NER-only timings, leave incremental client and semantic cost unresolved. RAPPOR establishes precedent for accounting for repeated randomized contributions [13]; PriVoke's ledger relies on retained state and a trusted clock, with its finite calculation also assuming independent uniform OS bits. Participation, delivery selection and metadata remain outside the content bound. No deployment-usefulness threshold was defined, and randomized aggregates provide no corrected contextual training labels. Reporting privacy, representative monitoring and useful supervision thus remain separate requirements.

The sealed capsule reproduced twenty-four invented semantic requests across six packages with matching identities and outcomes (Supporting Detail, Section I). This is local replay using existing host dependencies, not public Git-only installation, fresh-environment reproduction or a protected-final rerun. Ignored pinned assets and licensing constraints affect redistribution. Further evaluation should therefore target independent contextual adjudication, protected generalization, broader receiver coverage, baseline-relative semantic cost and human utility.

IX. Conclusion

PRIVOKE connects local prompt inspection, explicit request enforcement, version-bound guarded revision and separate randomized reporting in an implemented lifecycle. Its evaluation shows why those mechanisms need distinct acceptance evidence. Authored contextual agreement improved, yet action errors and poor specificity on separate targets remained. Installed-browser probes verified supported request handling, including original-content forwarding under WARN. Every matched full-model candidate improved aggregate agreement but failed retention, making average gain an insufficient basis for publication. Resource and telemetry studies quantified bounded costs and privacy behavior without demonstrating deployment utility. The contribution is an inspectable mechanism and evidence of its limits; independent contextual accuracy, useful adaptation without lost protection and human benefit remain open.

AI Assistance

AI assistance was used to produce provisional contextual labels and support manuscript drafting. GPT-6 Astra assisted manuscript review and rewriting. The provisional labels have not been established as independently adjudicated ground truth. Responsibility for the manuscript and its claims remains with the authors.

References

[1] B. B. Gupta, G. Akshat, V. Arya, A. Wadee, D. Alsalman, and P. Vijayakumar, "Enhancing user prompt confidentiality in large language models through advanced differential encryption," *Computers & Electrical Engineering*, vol. 116, pp. 109215–109215, 02 2024.

[2] W. Liang, Y. Zhang, M. Codreanu, J. Wang, H. Cao, and J. Zou, "The widespread adoption of large language model-assisted writing across society," *Patterns*, vol. 6, no. 12, p. 101366, 2025.

[3] T. Koraag, N. Wagner, F. Dobslaw, and L. Gren, "Comparing open-source and commercial llms for domain-specific analysis and reporting: Software engineering challenges and design trade-offs," *arXiv preprint arXiv:2509.24344*, 2025.

[4] S. Ray, "Samsung bans chatgpt among employees after sensitive code leak," *Forbes Business*, 05 2023.

[5] J. Dastin and A. Tong, "Focus: Google, one of ai's biggest backers, warns own staff about chatbots," *Reuters (Technology)*, 06 2023.

[6] H. Nissenbaum, "Privacy as contextual integrity," *Washington Law Review*, vol. 79, no. 1, pp. 119–158, 2004.

[7] S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia, and S. Soatto, "Positive-congruent training: Towards regression-free model updates," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14299–14308, 2021.

[8] C. J. Chong, C. Hou, Z. Yao, and S. M. S. Talebi, "Casper: Prompt sanitization for protecting user privacy in web-based large language models," 08 2024. Version 1, 13 August 2024; supplied 13-page PDF. Not the 2025 proceedings version.

[9] C. J. Chong, C. Hou, Z. Yao, and S. M. S. Talebi, "Casper: Prompt sanitization for protecting user privacy in web-based large language models," in *2025 IEEE 12th International Conference on Cyber Security and Cloud Computing (CSCloud)*, pp. 122–131, 2025. Related preprint: arXiv:2408.07004v1.

[10] Y. Chen, T. Li, H. Liu, and Y. Yu, "Hide and seek (has): A lightweight framework for prompt privacy protection," 09 2023.

[11] Z. Shen, Z. Xi, Y. He, W. Tong, J. Hua, and S. Zhong, "The fire thief is also the keeper: Balancing usability and privacy in prompts," 06 2024.

[12] E. Keles and S. Bahtiyar, "Promptshield: Policy-aware dlp framework for generative ai prompts," in *2025 10th International Conference on Computer Science and Engineering (UBMK)*, pp. 1537–1542, 2025. Distinct conference publication record; relationship to the 2026 thesis not established.

[13] Ú. Erlingsson, V. Pihur, and A. Korolova, "RAPPOR: Randomized aggregatable privacy-preserving ordinal response," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014.

[14] "Presidio: Customizing nlp models." Official project documentation; accessed 14 October 2026. Local baseline versions and configuration are pinned separately.

[15] "BERT Miniatures: google/bert_uncased_L-4_H-128_A-2 model card." Pinned revision c29bee83fc7f003ac8c5e6e135529da4ecddb7c3; Apache-2.0 declaration; accessed October 12, 2026.

[16] "BERT Miniatures: google/bert_uncased_L-2_H-256_A-4 model card." Pinned revision 4e937a8675e5afd9a4836735c186ec01695bc3ea; Apache-2.0 declaration; accessed October 12, 2026.

[17] "BERT Miniatures: google/bert_uncased_L-12_H-256_A-4 model card." Pinned revision 2ab9a0f41435a4d23d4cbc11cc3e7c922545f13d; Apache-2.0 declaration; accessed October 12, 2026.

[18] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," 2019.

[19] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[20] "PIIMB: PII Masking Benchmark." Pinned Hugging Face dataset card. Revision 4a13e9f; accessed 14 October 2026. Character-level zero-shot masking benchmark; CC BY-NC 4.0.

[21] R. Arpaly and Y. Birman, "Advpiibench: The adversarial collapse of pii detection in llm interactions." https://huggingface.co/datasets/roei-ar/AdvPIIBench, 2026. Dataset. Presented as a poster at the 35th USENIX Security Symposium. Pinned revision 02741d9f; accessed 14 October 2026.

[22] Explosion, "en_core_web_lg 3.7.1 model release." https://github.com/explosion/spacy-models/releases/tag/en_core_web_lg-3.7.1. Accessed 14 October 2026. MIT license; spaCy compatibility greater than or equal to 3.7.2 and less than 3.8.0.